```
epochs = 50000
replay_size = 9000
batch_size = 50
lr = 0.0005
gamma = 0.99
replay = deque(maxlen=replay_size)
5
opt = torch.optim.Adam(params=GWagent.parameters(),lr=lr)
eps = 0.5
update_freq = 100
for i in range(epochs):
    pred = GWagent(state)
    action = int(torch.argmax(pred).detach().numpy())
    if np.random.rand() < eps:
6
        action = int(torch.randint(0,5,size=(1,)).squeeze())
    action_d = action_map[action]
    state2, reward, done, info = env.step(action_d)
    reward = -0.01 if reward == 0 else reward
7
    state2 = prepare_state(state2)
    exp = (state,action,reward,state2,done)

    replay = update_replay(replay,exp,replay_size)
    if done:
        state = prepare_state(env.reset())
    else:
        state = state2
    if len(replay) > batch_size:

        opt.zero_grad()


state_batch,action_batch,reward_batch,state2_batch,done_batch =
     get_minibatch(replay,batch_size)

        q_pred = GWagent(state_batch).cpu()
        astar = torch.argmax(q_pred,dim=1)
        qs =

Tnet(state2_batch).gather(dim=1,index=astar.unsqueeze(dim=1)).squee
ze()

        targets =

get_qtarget_ddqn(qs.detach(),reward_batch.detach(),gamma,done_batch
)

        loss = lossfn(q_pred,targets.detach(),action_batch)
        loss.backward()
```